

1

useEffect

by Rahul Rajeevan

<#>

Identify the problem

```
function UserList() {
  const [users, setUsers] = useState([]);

  // 🤔 Where should we put this?
  // fetch("https://api.example.com/users")
  //   .then(res => res.json())
  //   .then(data => setUsers(data));

  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}
</li>)}
    </ul>
  );
}
```

“If I uncomment this fetch and put it directly here, in the component body, what problems do you think we might hit?”

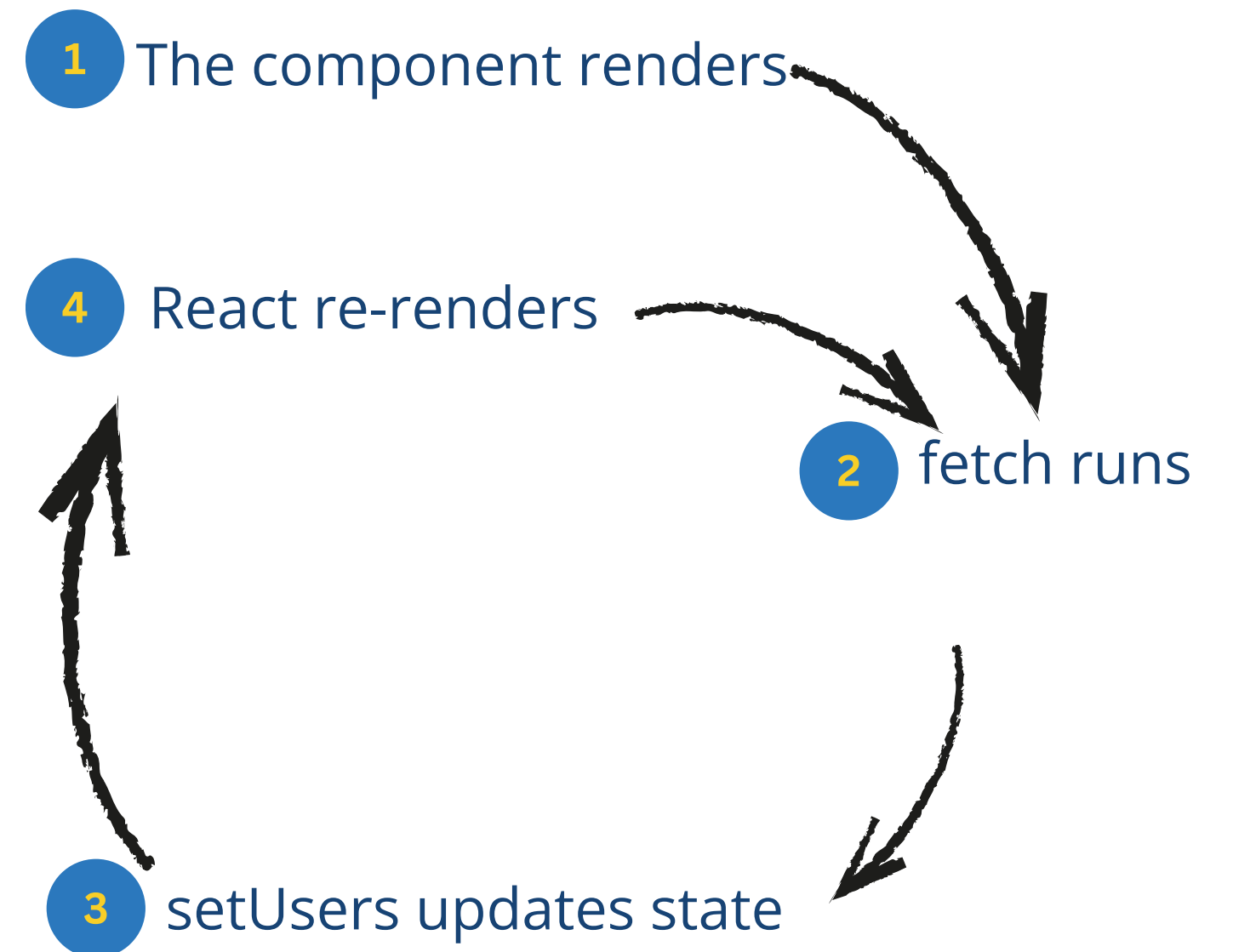
“Putting fetch in the component body means:

- it runs on every render,
- can easily cause **infinite loops** via setState,
- and breaks the rule that React’s render must be pure.

That’s why React gives us useEffect — a special place for side-effects, separate from the render itself.”

Infinite Loop with fetch and setState

```
function UserList() {  
  const [users, setUsers] = useState([]);  
  
  fetch("https://api.example.com/users")  
    .then(res => res.json())  
    .then(data => setUsers(data));  
  
  return (  
    <ul>  
      {users.map(u => <li key={u.id}>{u.name}</li>)}  
    </ul>  
  );  
}
```



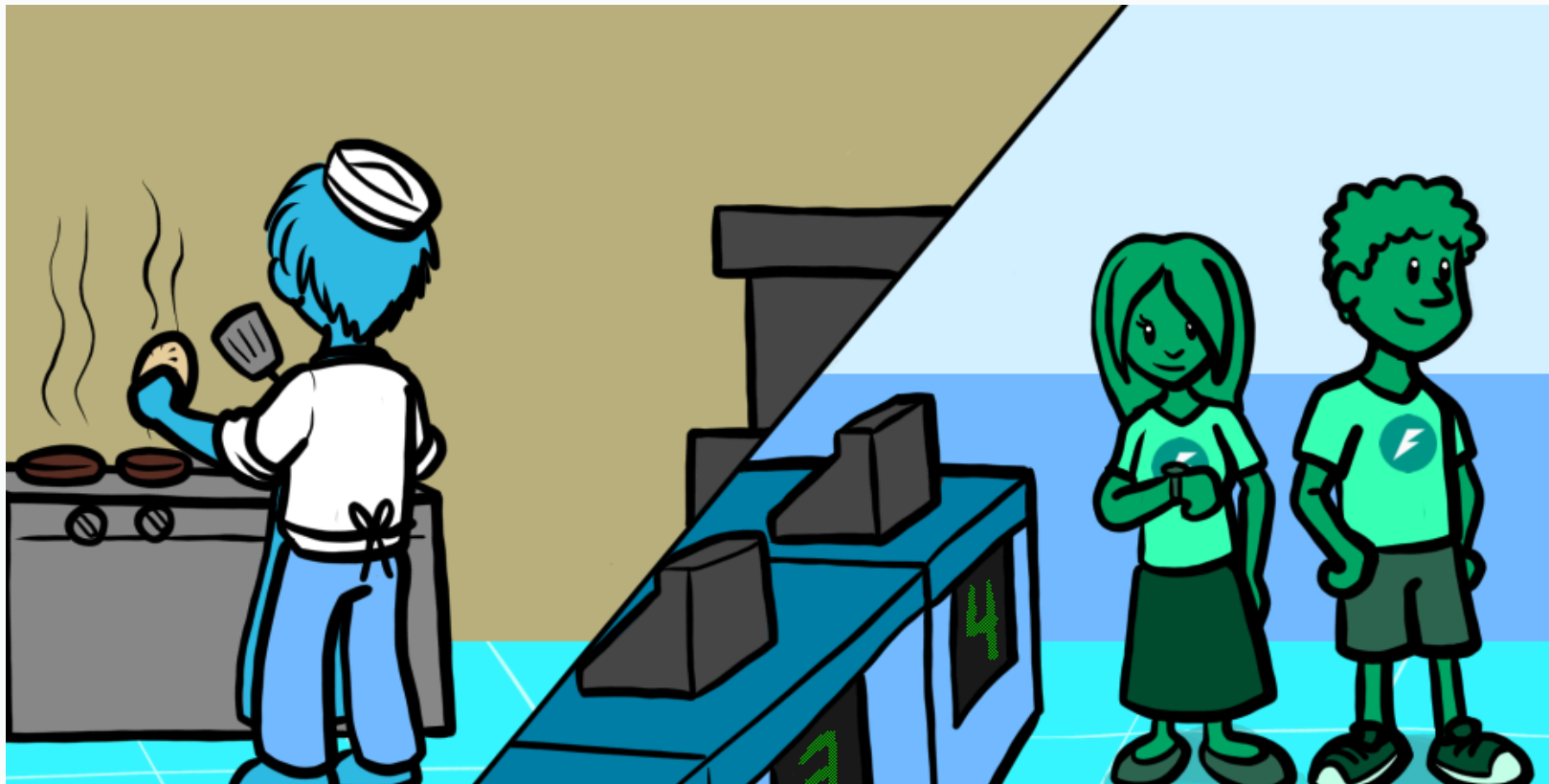


Introduction to `useEffect()`

Handling Side-Effects

Remember async operations in JS?

“Why did we need callbacks, promises, and async/await in the first place?”



Time taking operations

Async operations do not execute immediately, so waiting for them blocks the execution of other tasks.

We used promises to handle async ops

We used promises to handle async operations in JS. Here is an example:

```
1 function orderFood(order) {
2   return new Promise((resolve) => {
3     console.log(`Order placed: ${order}`);
4     setTimeout(() => {
5       resolve(`Order ready: ${order}`);
6     }, 3000); // Simulating food preparation time
7   });
8 }
9
10 console.log("Customer enters restaurant");
11
12 orderFood("Pizza").then((message) => {
13   console.log(message);
14 });
15
16 console.log("Other customers continue ordering...");
```

And then need to pass this orderFood() function to the event listener attached to an element.

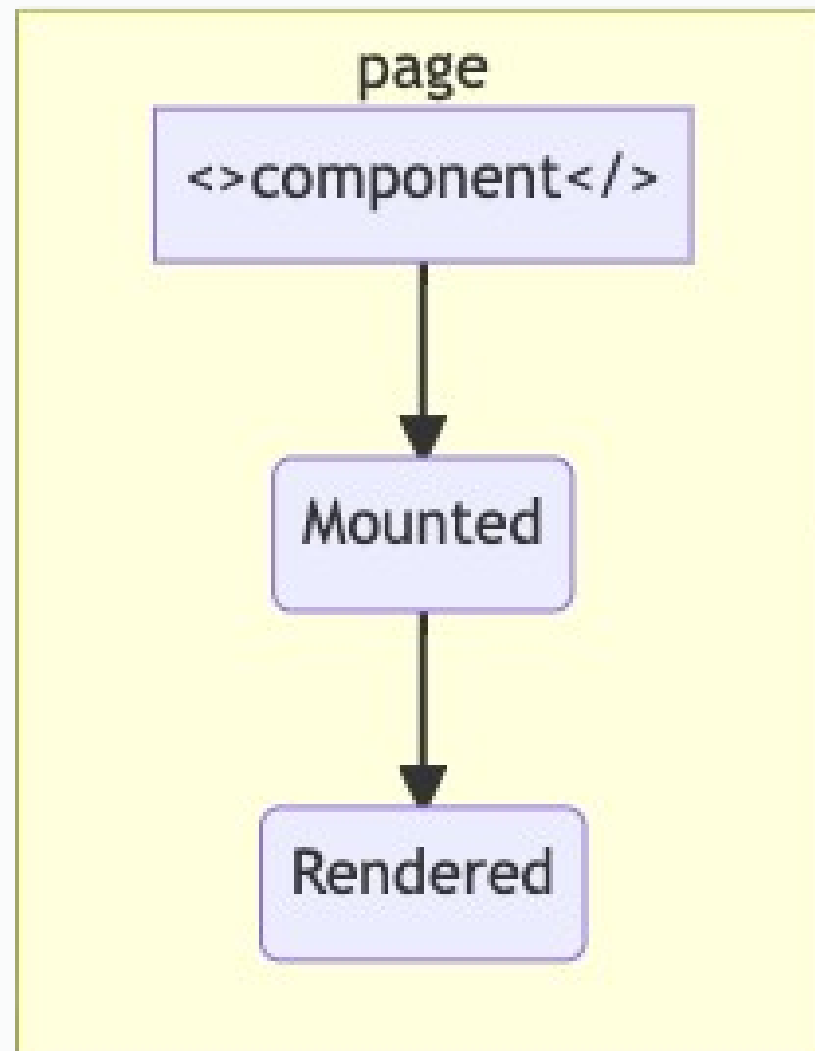
```
1 // Mimicking useEffect with an event listener
2 document.getElementById("orderBtn").addEventListener("click", () => {
3   orderFood("Pizza").then((message) => {
4     console.log(message);
5   });
6 });
7
8 console.log("Other customers continue ordering...");
9
10
11 <html>
12   ....
13   <body>
14     <button id="orderBtn">
15       Order Pizza
16     </button>
17   </body>
18 </html>
```

**Do I need to do all
this work to each
element we want
to listen?**



Don't worry we have `useEffect()` in React

Component Lifecycle



1. <Component /> (Component Created / Invoked)

- You write `<MyButton />` or `<UserCard />`
- React calls the function/class to generate JSX/HTML.

2. Mounted

- The component's HTML elements are created in **memory**.
- Then React inserts those elements into the DOM tree of the browser.
- So now the component officially exists in the DOM.

BUT — it is not yet painted on the screen. It's just been placed in the DOM structure.

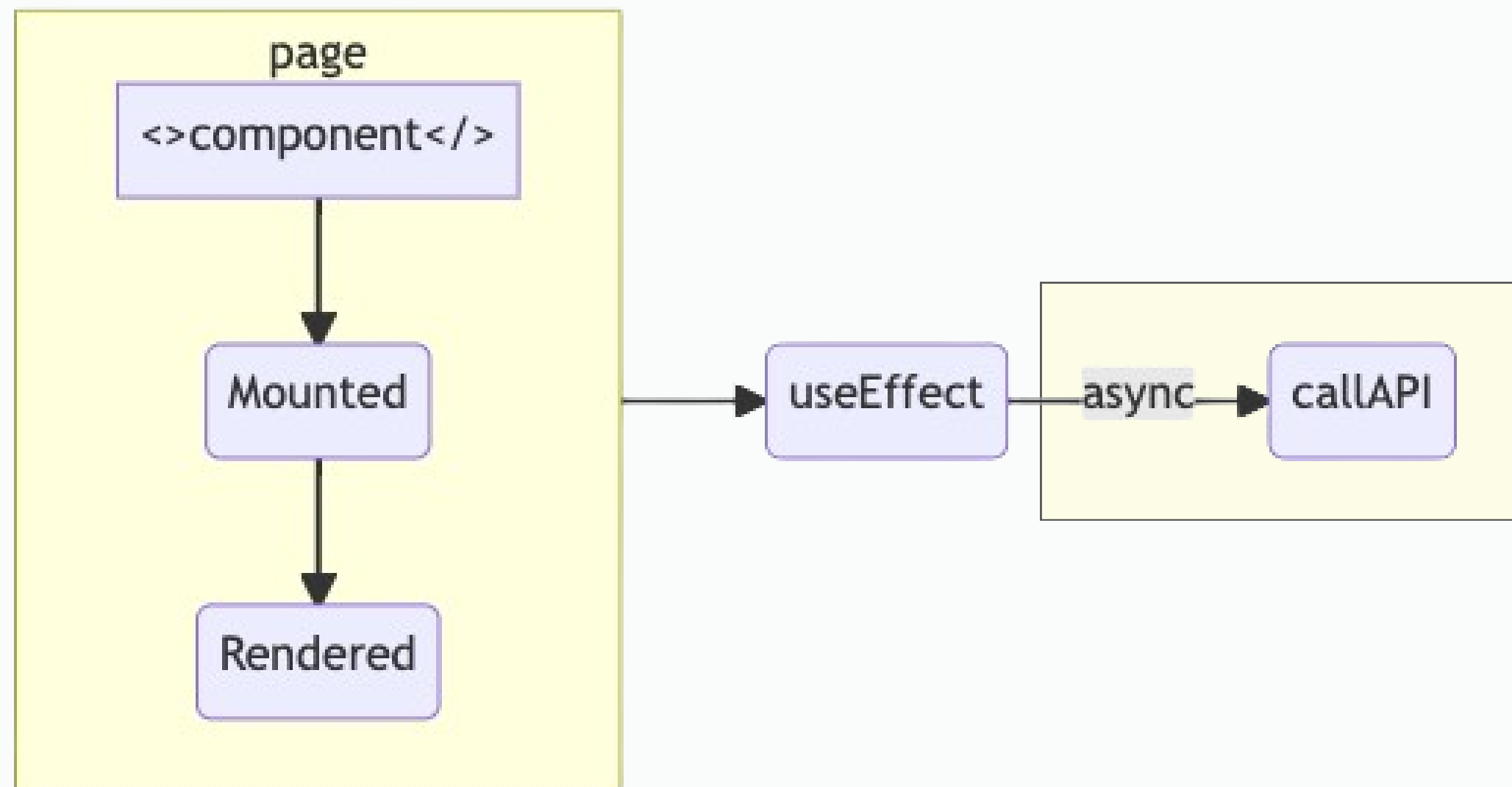
3. Rendered

The browser has read the DOM

- Applied CSS
- Calculated layout
- Painted pixels on the screen

Now the user can see the component.

`useEffect` in React handles **side effects** like data fetching, subscriptions, or DOM updates after render.



useEffect() simplifies page updates

Just like promises facilitates the async operations to happen in right order, useEffect makes sure that async code runs only at the end allowing other parts of the page to load immediately.

After the page rendering is complete useEffect runs, re-rendering the page if code inside the useEffect changes the content of the page.

What Are Side Effects in React?

In React, your component's job is only one thing:

Take data → return UI (JSX).



```
function Hello() {  
  return <h1>Hello World</h1>;  
}
```

Normal (No Side Effect)

It doesn't touch APIs, timers, or browser directly.

So — no side effects.

A side effect is anything a component does that:
is not about returning JSX
touches the “outside world”.



```
function User() {  
  console.log("Component rendered!");  
  return <h1>User</h1>;  
}
```

Side Effect Example

`console.log()` is not part of UI.

It interacts with the browser console.

→ That is a side effect.

What Is the “Outside World”?

Anything outside React’s pure rendering system.

1. API Calls (fetch, axios)

React does not control servers or networks → so fetching data is a side-effect.

```
useEffect(function () {  
  fetch("https://jsonplaceholder.typicode.com/posts/1")  
    .then(res => res.json())  
    .then(data => console.log(data));  
}, []);
```

2. Timers (setTimeout, setInterval)

React does not manage browser timers → creating one is a side effect.

```
useEffect(function () {  
  const id = setTimeout(function () {  
    console.log("Timer fired!");  
  }, 2000);  
  
  return function () {  
    clearTimeout(id);  
  };  
}, []);
```

3. Event listeners NOT created by JSX

JSX event listeners (like `onClick`) are inside React's system → not side effects.

But adding listeners directly to `window` or `document` is a side effect.

```
useEffect(function () {  
  function handleResize() {  
    console.log("Window resized");  
  }  
  window.addEventListener("resize", handleResize);  
  
  return function () {  
    window.removeEventListener("resize",  
handleResize);  
  };  
}, []);
```

4. Manual DOM changes

React wants to own the DOM.

If you mutate the DOM yourself → side effect.

```
useEffect(function () {  
  document.body.style.backgroundColor = "yellow";  
  
  return function () {  
    document.body.style.backgroundColor = "white";  
  };  
}, []);
```

5. Subscriptions (websocket, Firebase, etc)

React has no idea about your external subscription → side effect.

```
useEffect(function () {  
  const socket = new  
  WebSocket("ws://example.com");  
  
  socket.onmessage = function (event) {  
    console.log("New message:", event.data);  
  };  
  
  return function () {  
    socket.close();  
  };  
}, []);
```

6. Changing Browser Storage (localStorage / sessionStorage)

Browser storage lives outside React → so writing to it is a side effect.

```
useEffect(function () {  
  localStorage.setItem("theme", "dark");  
}, []);
```

Js vs React for fetch

Suppose you are provided a url from where you need to fetch the data and display its title in your web page.

```
1 document.addEventListener("DOMContentLoaded", () => {
2   const heading = document.createElement("h1");
3   heading.textContent = "Loading...";
4   document.body.appendChild(heading);
5
6   fetch("https://jsonplaceholder.typicode.com/todos/1")
7     .then(response => response.json())
8     .then(data => {
9       heading.textContent = data.title;
10      // Update content after fetching
11    });
12 });
```

If we use javascript we need to add event listener and create, insert dom elements by ourself

```
1 function App() {
2   const [data, setData] = useState(null);
3
4   useEffect(() => {
5     fetch("https://jsonplaceholder.typicode.com/todos/1")
6       .then((response) => response.json())
7       .then((json) => setData(json));
8   }, []); // Runs only after the initial render
9
10  return <h1>{data ? data.title : "Loading..."}</h1>;
11 }
12
13 export default App;
```

Whereas React runs useEffect at the end and if useEffect code changes content of the page, it reloads the latest copy of the page.

Order of execution...

Let's understand the order of execution of `useEffect()` with respect to rest of the code.

```
1  function App() {  
2    const [data, setData] = useState(null);  
3  
4    useEffect(() => {  
5      fetch("https://jsonplaceholder.typicode.com/todos/1")  
6        .then((response) => response.json())  
7        .then((json) => setData(json));  
8    }, []); // Runs only after the initial render  
9  
10   return <h1>{data ? data.title : "Loading..."}</h1>;  
11 }  
12  
13 export default App;
```

1. State Variable is declared and assigned *null*

3. Code inside `useEffect()` block executes

2. Expression is evaluated and since data is *null*. Loading... is returned

Anatomy of useEffect

useEffect takes 2 arguments. A callback function and a dependancy array (optional)



```
useEffect(callbackfunction, [dependancies]);
```

- **callback function**: a function where you put your side effect
- **deps (optional)**: array of values React watches
- React calls effectFn after it paints the UI to the screen.

Pattern 1 — useEffect on every render (no dependency array)

```
useEffect(() => {  
  console.log("Effect ran");  
});
```

When it runs

- Runs after the first render
- Runs after every re-render (whenever any state/props change)



“I don’t care what changed, just **run this effect** after every render.”

Example (for understanding, not best practice):

```
function Logger({ count }) {  
  const [name, setName] = useState("");  
  
  useEffect(() => {  
    console.log("Component rendered. Count:",  
count, "Name:", name);  
  });  
  
  return (  
    <>  
      <p>{count}</p>  
      <input value={name} onChange={e =>  
setName(e.target.value)} />  
    </>  
  );  
}
```

Every time count or name changes → re-render
→ effect runs.

Why this can be bad:

Easy to create infinite loops

```
useEffect(() => {  
  setCount(c => c + 1); // updates state →  
rerender → effect runs again...  
});
```

Pattern 2 — Empty dependancy array []

useEffect runs once on mount



```
useEffect(() => {  
  console.log("Runs only once when component  
mounts");  
}, []);
```

Common use cases :

- Initial data fetch
- One-time subscriptions / listeners
- Initialize 3rd-party libraries (charts, maps, sliders)

When it runs

After the first render only

React treats [] as: "this effect doesn't depend on any reactive values"

Pattern 3 — Non empty dependancy array

useEffect runs when some values change ([deps...])



```
useEffect(() => {  
  console.log("Effect ran because count or userId  
changed");  
}, [count, userId]);
```

Common use cases :

This is the most important way to use useEffect

Think of it like:

“Whenever X changes, do Y.”

Where X is one or more dependencies.

When it runs

- After first render
- Then, after any render where at least one dependency value changed
- (by === comparison: for objects/arrays, reference change matters)

Key Takeaways

- `useEffect` handles side-effects
- Dependency array controls execution
- Cleanup prevents memory leaks
- Effects connect React to real-world behavior

In Class Questions

**Thanks
for
watching!**

Feedback

Now it's time to give your valuable feedback

